

Four-Cycle Detection Analysis Paper

V Tovmasian, E Plata, A Mori-Abrams, J Ata-Quiroz

April 2026

1 CoverPage

E Plata AI use: Polish and breaking down the theorem lemmas into more basic terms for understanding

A Mori-Abrams: Used AI for understanding different algorithms, helped fix up and properly word my section were the Problem Motivation and History

V Tovmasian: AI used for finding research papers and helping Latex code

J Ata-Quiroz: Breaking down complex topics in the paper into manageable sections, and searching for related algorithms

2 Problem

In this paper, we study the induced 4-cycle detection problem [3]. The input is an undirected graph $G = (V, E)$ with n vertices [2]. The desired output tells us if the graph contains a 4-cycle induced graph such that G contains 4 vertices a, b, c, d where $(a, b), (b, c), (c, d), (d, a) \in E$ but $(a, c) \notin E$ and $(b, d) \notin E$ [cite: 4]. The four vertices form a square 4-cycle, but there cannot be any diagonal edges, which makes this 4-cycle problem an induced 4-cycle problem [6].

The paper defines an induced 4-cycle as a tuple where $(w, x), (x, y), (y, z), (z, w)$ are edges and $(w, y), (x, z)$ are not [4].

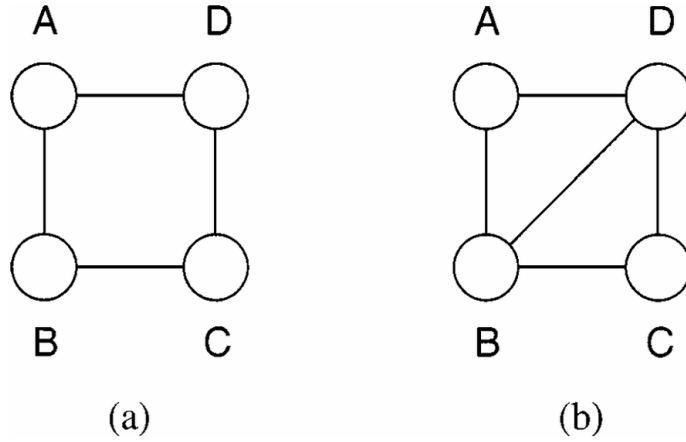


Figure 1: Induced(a) vs Not induced(b) [4]

In Figure 1, we can see the slight difference between these 4-cycles. The graph labeled “a” is a perfect example of an induced 4-cycle graph. Nodes are only connected to perimeter edges only, where graph “b” has a diagonal edge (b, d) , which breaks the induced condition.

Detecting if a 4-cycle graph is induced is harder than checking if it is non-induced. Since if G contains any 4-cycle regardless if there are any diagonals is solvable in $O(n^2)$ time as stated in the paper, “where detecting a (non-induced) 4-cycle has a classic $O(n^2)$ time combinatorial algorithm” [1].

The run time is only $O(n^2)$ since the algorithm just has to check if for every pair of vertices (u, v) , they share at least 2 common neighbors. This works because we only have to check if certain edges exist. But it is not that simple with an induced 4-cycle since we need to check if the 4 edges on the perimeter exist, like what we did with non-induced to check for a 4-cycle, but we also have to make sure that the two diagonal edges are also absent.

Checking to see if an edge is present is not the same as checking if they are missing. This is why the fastest known algorithm noted in this paper states

a running time of $O(n^\omega)$ time, where $\omega < 2.3713$ is the matrix multiplication exponent, which is slower than non-induced 4-cycle detection $O(n^2)$.

I_4 	K_4 	$n^{\omega(2,1,1)}$	$n^{\omega(2,1,1)}$
co-diamond 	diamond 	n^ω	n^ω
co-paw 	paw 	n^ω	n^ω
$2K_2$ 	C_4 	n^ω	$m + n$ $m^{4/3}$ or n^2 [new for C_4]
claw 	co-claw 	n^ω	n^ω
P_4 		$m + n \leq n^2$	$m + n$

Figure 2: Eleven possible induced 4-vertex subgraph patterns and their best known algorithms and conditional lower bounds [3]

In Figure 2, we can see the different patterns of induced 4-cycles, as well as their best known running times and best known lower bound. C_4 and $2K_2$ (complement of C_4) are the only patterns without a tight bound. That is a gap between the bounds, which is what we will be studying in this paper.

So if there is already a $O(n^\omega)$ running time, why does this paper want to solve this problem combinatorially? The next section addresses this.

3 Motivation

The fastest known algorithm to solve induced C_4 detection runs in $O(n^\omega)$ using fast matrix multiplication. But the paper argues that a combinatorial algorithm is worth finding. As they noted on page 3, “The term ‘combinatorial’ intuitively refers to algorithms that avoid the use of fast matrix multiplication” [1]. There are 2 reasons for solving this problem combinatorially.

The practical reason is that fast matrix multiplication algorithms are known to be impractical for real-world use. Strassen’s $O(n^{2.81})$ algorithm has poor

cache locality and requires manipulating large numbers, and it exploits cancellations using algebraic tricks that are hard to follow — all we really know is that the algorithm works. Every faster algorithm pushing ω closer to 2 is galactic, meaning the constant factors are so large the algorithm only beats $O(n^3)$ on inputs too large to ever be used in any real world scenarios [2]. Any real system that needs to detect induced C_4 cannot use these algorithms.

The theoretical reason is that algebraic methods provably cannot solve several major open problems. AFK+24 lists four [2]: all-pairs shortest paths uses min instead of subtraction, and min has no inverse, so Strassen’s cancellation tricks don’t work. Online matrix-vector multiplication requires seeing the whole input upfront, but Strassen’s formulas need future columns to compute earlier answers. Hyperclique detection has been shown to lack the algebraic structure that Strassen-style methods require. Lastly, triangle enumeration loses witness information when cancellations destroy intermediate data. Progress on these problems requires combinatorial techniques.

The paper we study contributes directly to this goal — giving the first truly subcubic combinatorial algorithm for induced C_4 detection, closing a gap that had resisted combinatorial approaches until now.

4 History

The induced C_4 detection problem has a long history of incremental improvements. Progress is split into two categories: algorithms designed specifically for induced C_4 , and the broader effort to find fast combinatorial algorithms for Boolean Matrix Multiplication (BMM), which is closely related to induced C_4 detection.

Induced C_4 algorithms. The naive brute force approach enumerates all four-vertex subsets and checks each one, giving a running time of $O(n^4)$. A simple improvement reduces this to $O(n^3)$: for every non-edge (a, c) , scan their common neighbors to see if they have a non-adjacent pair (b, d) . If such a pair exists, vertices $\{a, b, c, d\}$ form an induced C_4 . The current best known algorithm uses fast matrix multiplication to achieve $O(n^\omega)$ time, where $\omega < 2.3713$. Both the $O(n^3)$ combinatorial bound and the $O(n^\omega)$ algebraic bound were established by Vassilevska Williams, Wang, Williams, and Yu in 2014 [5]. As discussed in the Motivation section, the $O(n^\omega)$ algorithm relies on algebraic methods and is impractical for real-world use.

Combinatorial BMM. This second track tries to find a fast combinatorial algorithm for Boolean Matrix Multiplication, since a faster combinatorial BMM algorithm would imply faster combinatorial algorithms for many graph problems, including induced C_4 . The history of improvements starts at the trivial $O(n^3)$ bound and improves by log factors over many years. Arlazarov et al. (1970) achieved $O(n^3/\log^2 n)$ using the “Four Russians” technique, Bansal and Williams (2009) improved this to $O(n^3/\log^{2.25} n)$, Chan (2015) reached

$O(n^3/\log^3 n)$, and Yu (2015) reached $O(n^3/\log^4 n)$ [6]. None of these are truly subcubic — they only shave off polylogarithmic factors.

In 2024, the polylog barrier was broken: AFK+24 gave a combinatorial BMM algorithm running in $n^3/2^{\Omega(\sqrt{\log n})}$ [2]. Although this still falls short of being truly subcubic in the strict sense, this was a speedup stronger than any polylog factor. The breakthrough introduced a new graph regularity decomposition technique — a method that partitions a graph into highly structured pieces (large cliques) plus a sparse remainder, allowing the algorithm to analyze each piece more efficiently than the graph as a whole.

The paper we study extends this decomposition approach to the induced C_4 problem, producing the first truly subcubic combinatorial algorithm for induced C_4 detection, with a running time of $O(n^{2.84})$ [1]. While this is still slower than the $O(n^\omega)$ algebraic algorithm, it is the first to break the cubic barrier without relying on fast matrix multiplication, demonstrating that decomposition-based techniques can be pushed into territory previously thought to require algebraic tools.

5 Algorithm

Several algorithms for induced 4-cycle detection have achieved a cubic running time complexity. The algorithm to be analyzed in this section was introduced by Amir Abboud, Shyan Akmal, and Nick Fischer in their paper *A Truly Subcubic Combinatorial Algorithm for Induced 4-Cycle Detection*, wherein they discuss their new take on an algorithm that may achieve sub-cubic complexity under notable limitations. The algorithm that they have come up with the following:

Theorem 1. *There is a deterministic, combinatorial algorithm solving induced 4-cycle detection on graphs with n vertices in $\tilde{O}(n^{3-1/6}) \leq O(n^{2.84})$ time.*

Proof. Apply Theorem 30 with parameters $L = \lfloor (1/2) \log n \rfloor$ and $H = \lfloor \log n \rfloor$. This takes

$$\tilde{O}(n^2 \cdot 2^L + n^3/2^L) \leq \tilde{O}(n^{5/2})$$

The algorithm that they have constructed relies on multiple theorems and lemmas that are directly relevant to the algorithm at hand, each with their own lemmas that are crucial to proving the main algorithm. We will discuss the main theories and lemmas that are tied to the final algorithm that they constructed within their paper as simply as possible within the bounds of this paper.

5.1 Theorem 30 (Layered Cluster Decomposition)

The first theorem utilized in the sub-cubic algorithm discussed revolves around what is labeled as Theorem 30, which introduces the concept of layered cluster decomposition wherein a graph's vertex set V is partitioned into clusters, or

cliques, across many levels. The theorem serves as the foundation for the algorithm and transforms a graph into a structured hierarchy that we can analyze efficiently.

Theorem 30 (Layered Cluster Decomposition). *Let $G = (V, E)$ be a graph, and let L and H be integers with $1 \leq L \leq H = \lfloor \log n \rfloor$. There is an $\tilde{O}(n^2 \cdot 2^L + n^3/2^L)$ -time algorithm that either reports an induced 4-cycle in G , or returns*

- a vertex partition $V = V_L \sqcup \dots \sqcup V_H$,
- collections of disjoint vertex subsets $\mathcal{X}_L, \dots, \mathcal{X}_H$, and
- the sets $N_\ell(x, y) = N_{V_\ell}(x) \cap N_{V_\ell}(y)$ for all $\ell \in \{L+1, \dots, H\}$ and non-edges $(x, y) \notin E$,

such that we have

Levels of Clusters: all parts $V_L, \dots, V_H$ can be further decomposed into $V_\ell = \bigsqcup_{X \in \mathcal{X}_\ell} X$, where each vertex subset $X \in \mathcal{X}_\ell$ is a clique in G of size $\Theta(n/2^\ell)$; and

Bounded Common Neighborhoods: $|N_\ell(x, y)| \leq O(n/2^\ell)$ for all $L < \ell \leq H$ and $(x, y) \notin E$.

What this theorem guarantees is that given a graph $G = (V, E)$, the vertices V are partitioned into levels, where at each level the vertices are organized into clusters. The higher the level, the smaller the cluster, and the lower the level, the larger the cluster. Then, for any two non-adjacent vertices (x, y) , the set $N_\ell(x, y)$ represents the common neighbors in a cluster t_ℓ . The goal of which is to bound common neighborhoods where the structure constructed by the theorem—the number of common neighbors of (x, y) in cluster t_ℓ —provides at most $O(n^2 t_\ell)$. This serves to bound the later lemmas within the algorithm to achieve sub-cubic runtimes.

5.2 Lemma 31 (2-Cluster Detection)

“There is an $O(n^2)$ -time algorithm that detects a 2-clustered induced 4-cycle in G if any exist, and otherwise returns concise orderings for all pairs of clusters”

$$\mathcal{X} = \bigsqcup_{\ell=L}^H \mathcal{X}_\ell$$

Lemma 31 iterates over all distinct pairs of clusters across every level L to H in the decomposition. This checks each pair for an induced 4-cycle whose vertices are all sourced in, at most, two distinct clusters (a, b) . If there is a detected induced 4-cycle, it is reported. However, if no such cycle is detected then it serves that there is not a 2-clustered 4-cycle. This returns a precise ordering and relationship between clusters, and passes this information to Lemma 36 for reference to aid in efficiency and detection.

5.3 Lemma 36 (3-Cluster Detection)

Given orderings between all cluster pairs in G , there is an $\tilde{O}(n^{5/2})$ time algorithm that determines if G contains a 3-clustered induced 4-cycle.

Lemma 36 searches for an induced 4-cycle whose vertices span 3 distinct clusters. It does this by separating the clusters into three distinct types t_1, t_2, t_3 to represent the level index that the clusters belong to. This takes every possible combination of types that each cluster whose cycle vertices may belong to. Each case is designed so that each possible type falls into one case to ensure correctness.

Case 1 is applied when both t_2 and t_3 are above the minimum level L (assuming $t_3 \geq t_2$ and $t_{2,3} > L + 1$), where high level indexes correspond to smaller clusters which can be used to detect a cycle in $O(n^{5/2})$ time. This is done by Lemma 33, which builds sets of clusters containing valid 2-paths from both directions and bounds the number of induced paths using Theorem 30’s bounds.

1. If $t_2 \geq L + 1$: In this case, we also have $t_3 \geq t_2 \geq L + 1$. Thus we can apply the algorithm from Lemma 33 to detect an induced 4-cycle of type $\tilde{t}$ in

$$O(n^3 / 2^{\min(t_2, t_3)}) \leq O(n^3 / 2^L) \leq O(n^{5/2})$$

time.

Case 2 initiates when Case 1 fails and takes the sum of types t_1 and t_3 . This assumes that the sum is larger than $3/2 \log(n)$ and forces each above the

minimum level allowing the desired cycle to be detected in $O(n^{5/2})$ time similar to Case 1. This is done through Lemma 35, which handles the heavier positions t_1 and t_3 , enumerating the common neighbors for 4-cycle detection.

2. If $t_1 + t_3 > (3/2) \log n$: Since $L \leq (1/2) \log n$ and $H \leq \log n$, in this case we have $t_1, t_3 \geq L+1$. Thus, we can apply Lemma 35 to detect an induced 4-cycle of type $\vec{t}$ in

$$O(n^4 / 2^{t_1+t_3}) \leq O(n^4 / 2^{(3/2) \log n}) \leq O(n^{5/2})$$

time.

Case 3 is the final case and is executed when the previous two fail. The sum of all three types of clusters are subtracted from the minimum of the level index across the three. This keeps the bounds to $3/2 \log(n)$ threshold, meaning that the levels the clusters exist on are still relatively large and have small cliques. This is then passed to Lemma 32 which handles this by using the structural information that was computed earlier and bounds common neighborhoods from Theorem 30 to detect a 4-cycle in $O(n^{5/2})$ time.

3. If $t_1 + t_2 + t_3 - \min(t_1, t_2, t_3) \leq (3/2) \log n$: In this case, the algorithm from Lemma 32 detects an induced 4-cycle of type $\vec{t}$ in

$$\tilde{O}(n \cdot 2^{t_1+t_2+t_3-\min(t_1,t_2,t_3)}) \leq \tilde{O}(n \cdot 2^{(3/2) \log n}) \leq \tilde{O}(n^{5/2})$$

time.

5.4 Lemma 42 (4-Cluster Detection)

Lemma 42 is the final lemma within the overarching algorithm and is observed when no 2-clustered nor 3-clustered induced 4-cycle exists, meaning that the last possible option is that it spans four distinct clusters. If this is the case, four categories of clusters are made such that t_1 is the smallest of the four. This algorithm tries all possible types of clusters with four cases being checked:

Case 1 is applied when the sum of all types minus the minimum (t_1) is less than $11/6 \log(n)$, which keeps the level sum small enough for Lemma 37, which tries all cluster combinations and pairs in $O(n^{17/6})$.

Case 2 is observed when Case 1 fails, checking two clusters that are opposites (i.e., clusters that have 2 degrees of separation). The sum of the two opposite clusters is greater than $11/6 \log(n)$ since Case 1 failed. This forces both clusters to be greater than $L + 1$, allowing other lemmas (38 and 39) to enumerate on all n^2 pairs possible and find common neighbors before testing for an induced 4-cycle.

Case 3 is executed if Case 2 fails and applies if there exists a level gap of at least $1/6 \log(n)$ between opposite clusters ($t_1 \leq t_3 - 1/6 \log(n)$ or $t_2 \leq$

$t_4 - 1/6 \log(n)$). This is used to break down the structure of the neighborhoods and bounds the search for the 4-cycle by grounding the search based on the smaller cluster's size and utilizing Lemmas 40 and 41 to iterate through every possible pair to see if an induced 4-cycle exists in $O(n^{17/6})$ time.

Case 4 is executed when all other cases fail and is triggered when the sum of each opposite cluster pair exceeds the $7/6 \log(n)$ bounds and the gap is large. Since Case 4 is being executed, it implies that $t_1 \geq L + 1$ and as such, exceeds the minimum level established in Theorem 30, meaning we can reuse Lemmas 38 and 39 to iterate through all possible pairs, finding common neighbors, and testing for a 4-cycle in $O(n^{17/6})$.

5.5 Runtime

The total runtime for their final algorithm is an estimated $O(n^{2.84})$ or $O(n^{17/6})$ since, in the worst case possible, the algorithm runs and relies on Lemma 42 to detect an induced 4-cycle.

6 Implementation

For finding Implementation of this algorithm we choose to show the most difficult part to implement as the paper itself did not show a full explanation of Decomposition. This code was found on the internet and there are many others that show some form of Decomposition, but none that actually implement the exact algorithm that is used in this paper due to the novelty and difficulty of what is discussed in this paper. This code uses Python and specifically shows Clique decomposition using two different methods, one is Brute force the other is Greedy. This is a link to the repository where there is more of the code and has code to visualize the result for easy use. To use this code download the whole file, and run through Main. This is python code so have python installed on your machine, in the main is an already pre made test to show all you have to do is un-comment the Clique detection system you want and run it. The images below are showings of the code, one is of the main and another is of the Greedy implementation for finding Cliques.

```

import time
import pprint
import math
from benchmark import Benchmark
from gui import GUI
from generator import GraphGenerator
from clique import Clique
from graph import Graph

def main():
    # Graph
    g = GUI()
    a = GraphGenerator.generate(0, 10, 40)

    g.addGraph(a, (255, 0, 0), (255, 255, 0))

    # q, _ = Clique.findMaxBruteForceUp(a)
    # g.addGraph(q, (0, 0, 255), (0, 255, 255))

    # q, _ = Clique.findMaxBruteForceDown(a)
    # g.addGraph(q, (0, 0, 255), (0, 255, 255))

    q, _ = Clique.findMaxGreedy(a)
    g.addGraph(q, (0, 0, 255), (0, 255, 255))

    g.run()

if __name__ == "__main__":
    main()

    def findMaxGreedy(g):
        # Use the list of adjacencies as a starting point
        cliques = []
        for e in g.edges:
            base = e.v.copy()
            toTest = []
            for v in g.vertices:
                if not v in base:
                    toTest.append(v)
            cliques.append(CliqueGreedy(base, toTest))

    operations = 0
    size = 2

    while len(cliques) > 1 and size < len(g.vertices):
        # Check if the cliques can be expanded further

```

```

for clique in cliques:
    # Stop if the clique is the last one it is the maximum already
    if len(cliques) == 1:
        break

    # If the clique can be expanded
    cliqueWasExpanded = False

    # Iterate through all other vertices that are not in the clique
    for newVert in clique.toTest:
        # Remove from list of vertices to be tested
        clique.toTest.remove(newVert)

        # Check if it has connectivity to the already existing vertices in the set
        hasFullConnect = True
        for vert in clique.vertices:
            operations += 1
            if not g.edgeExists(newVert, vert):
                hasFullConnect = False
                break

        # Add to the list of vertices
        if hasFullConnect:
            clique.vertices.append(newVert)
            cliqueWasExpanded = True
            break

    # Discard any cliques that could not be expanded further
    if not cliqueWasExpanded:
        cliques.remove(clique)

size += 1

if len(cliques) > 0:
    return Clique.buildClique(cliques[0].vertices), operations

return None, operations

```

References

- [1] Fischer Abboud, Akmal. A truly subcubic combinatorial algorithm for induced 4-cycle detection. *Proceedings of the 2026 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2026.
- [2] Kelley Abboud, Fischer. New graph decompositions and combinatorial boolean matrix multiplication algorithms. *STOC 2024: Proceedings of the 56th Annual ACM Symposium on Theory of Computing*, 2024.
- [3] Williams Dalirrooyfard. Induced cycles and paths are harder than you think. *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, 2022.
- [4] Gharhramani Jordan. An introduction to variational methods for graphical models. *NATO ASI Series ((ASID, volume 89))*, 1999.
- [5] Yu Williams. Finding four-node subgraphs in triangle time. In *Proceedings of the Twenty-Sixth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1671–1680. SIAM, 2015.
- [6] Huacheng Yu. An improved combinatorial algorithm for boolean matrix multiplication. *ICALP 2015*, 2015.